

The spike detector worked exactly as documented. That was the problem.

What 6,720 smart-meter readings
said about a rule nobody had audited.

6,720

readings analysed

20

households, 5 regions

7 days

January 2024

The utility flags a spike when a home uses **more than 1.4× its own baseline**.

Simple, explainable, and it runs on every half-hourly reading in the fleet. In one week it raised **114 spikes**, alerted on **111** of them in a median of **14.4 minutes**, and closed **77** cases.

So I rebuilt the rule from the raw readings to check it. Three things fell out.

97.4%

of spikes alerted

14.4min

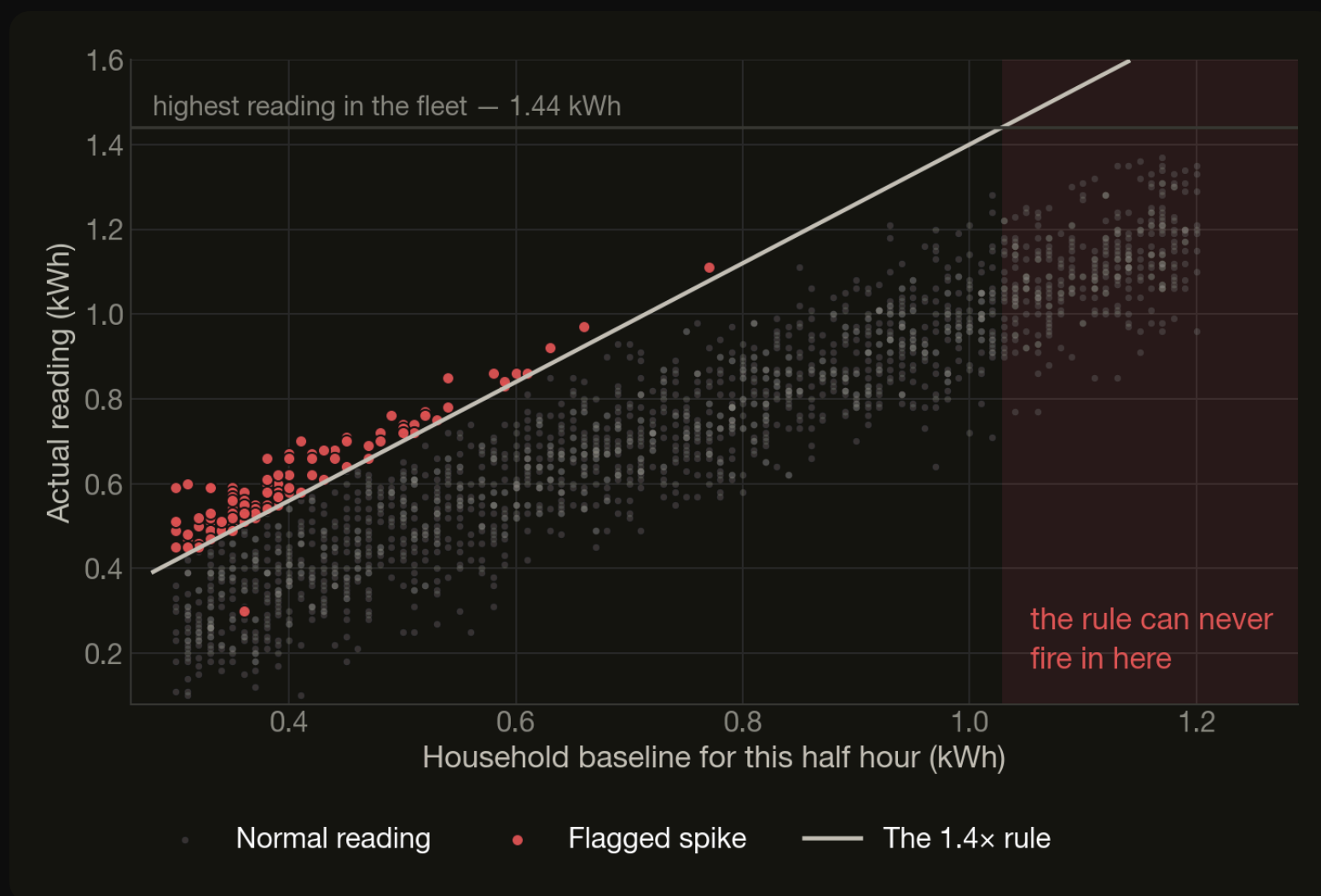
median alert latency

0

false alarms

Every spike it has **ever** raised came from a low-consumption home.

Readings top out at **1.44 kWh**. So wherever a household's baseline is above **1.03 kWh**, **$1.4 \times \text{baseline}$** is higher than anything a meter can physically report — and the rule cannot fire at all.



19.0%

of all readings sit in a band where the rule is **arithmetically unable** to fire.

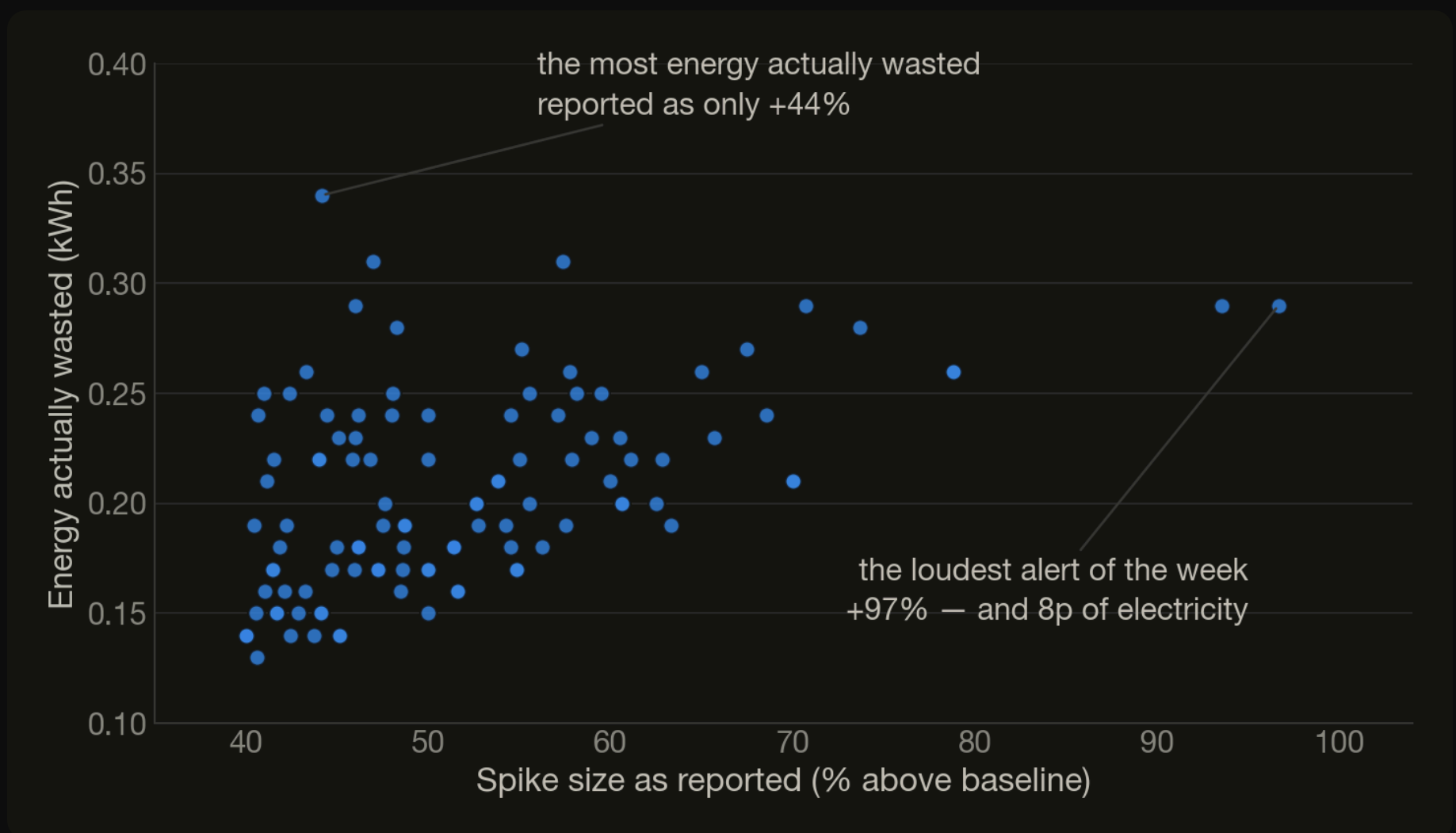
The highest baseline that has ever produced a spike is **0.77 kWh**.

The fleet median is **0.74 kWh**.

The detector isn't sampling the fleet. It's sampling the bottom half of it — and a faulty heater in a large house is invisible to it.

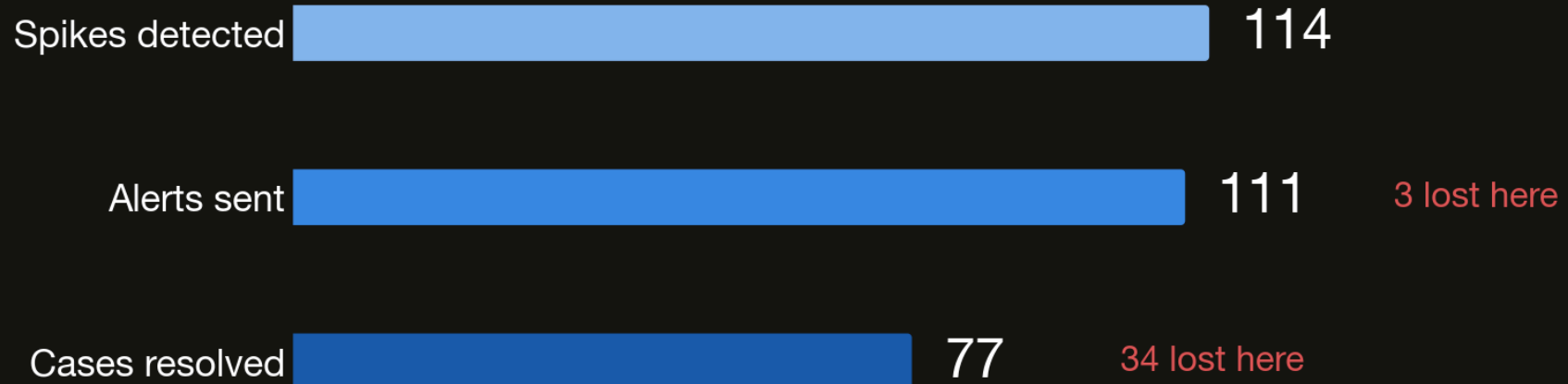
A "+97% spike" turned out to be **eight pence** of electricity.

Every flagged spike wasted between **0.13 and 0.34 kWh**. The headline percentage mostly records how small the baseline was — not how much energy went anywhere. Meanwhile **122 readings that were never flagged** wasted more than the median spike did.



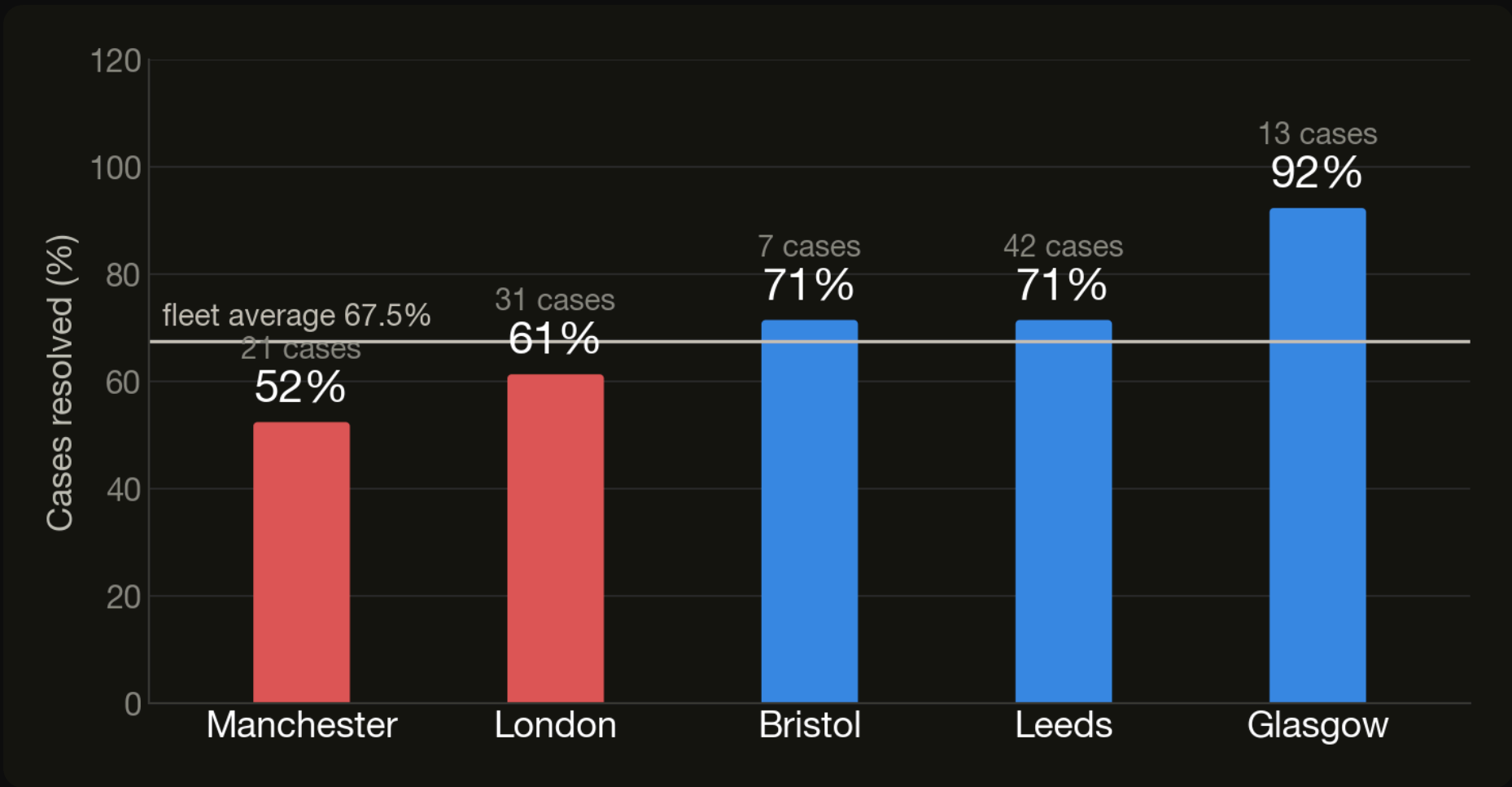
Detection loses 3%. The **operation** loses 30%.

114 spikes detected, 111 alerted, **77 resolved**. Everything that goes missing, goes missing after the alert lands — which makes it a process problem, not an analytics one.



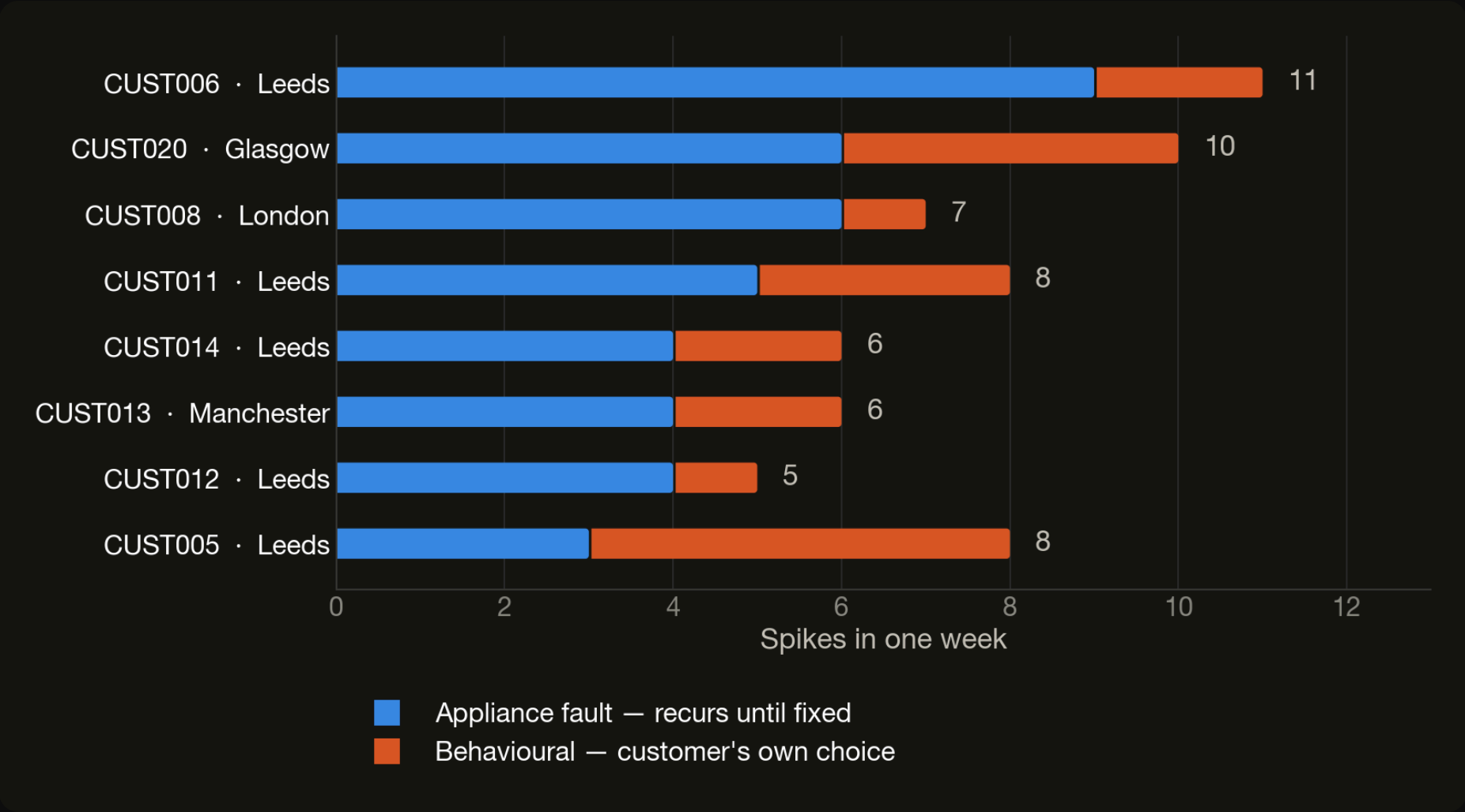
Same detector. Same alert. Same 14-minute latency.

Manchester closes half its cases; Glasgow closes nine in ten — and Manchester has the **fastest** alerts of any region, on a smaller caseload than Leeds. Nothing in the analytics explains the gap.



One broken appliance. Seven separate alerts.

The worst household logged 11 spikes — 7 of them the same reason code, on seven different nights. That isn't seven incidents, and seven notifications isn't proactive service. Alert on the event; escalate on the pattern.



The fix is **one line**.

```
# today  
usage > 1.4 * baseline  
  
# add  
or (usage - baseline) > 0.30
```

Ratio catches proportionally large events on small baselines.
Absolute excess catches materially large events on **any** baseline.
Run both.

In this week alone it surfaces **five events the rule ignored** — each wasting more energy than 97% of the spikes it did flag.

What I actually took from this one.

1 Audit the rule, not just the output.

113 of 114 flags reproduced exactly — and the one that didn't was flagged while sitting 17% *below* baseline.

2 A threshold can be well chosen and still be the wrong shape.

1.4 sits right at the tail of normal variation. The multiplier was fine. The multiplication wasn't.

3 Report the finding that isn't there.

No peak-hour concentration, no housing-type effect. Saying so is worth more than a chart that implies otherwise.

Built with Python, pandas and a self-contained HTML dashboard — no Tableau licence needed. Happy to share the write-up.